

JAVA COLLECTION FRAMEWORK – 12 PROGRAMS

Complete Practical Record – Part 2

Aim, Algorithm, UML Class Diagram, Source Code, Execution Commands, Output Screenshot and Result.

Program 1: ArrayDeque

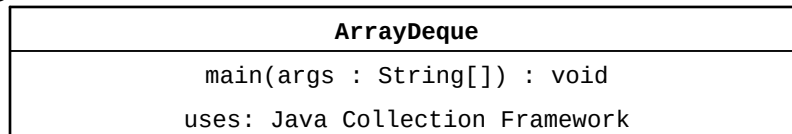
Aim / Objective

To demonstrate insertion, retrieval, and removal operations of ArrayDeque.

Algorithm / Logic

1. Create an ArrayDeque of String values.
2. Insert elements at the front using addFirst().
3. Insert elements at the rear using addLast().
4. Use offerFirst() and offerLast() to insert elements.
5. View the first and last elements using peekFirst() and peekLast().
6. Remove elements using pollFirst() and pollLast().

UML Class Diagram



Source Code

```
import java.util.ArrayDeque;

class ArrayDeque {

    public static void main(String[] args) {

        // Creating an ArrayDeque
        ArrayDeque<String> names = new ArrayDeque<>();

        // addFirst(E e)
        names.addFirst("Rahul");
        names.addFirst("Priya");

        System.out.println("After addFirst(): " + names);

        // addLast(E e)
        names.addLast("Arjun");
        names.addLast("Sneha");

        System.out.println("After addLast(): " + names);

        // offerFirst(E e)
        names.offerFirst("Kiran");
        System.out.println("After offerFirst(): " + names);

        // offerLast(E e)
        names.offerLast("Meena");
        System.out.println("After offerLast(): " + names);

        // peekFirst()
        System.out.println("First element using peekFirst(): "
            + names.peekFirst());

        // peekLast()
        System.out.println("Last element using peekLast(): "
            + names.peekLast());

        // pollFirst()
        System.out.println("Removed first element using pollFirst(): "
            + names.pollFirst());

        System.out.println("After pollFirst(): " + names);

        // pollLast()
        System.out.println("Removed last element using pollLast(): "
            + names.pollLast());
        System.out.println("After pollLast(): " + names);
    }
}
```

```
}  
}
```

Compilation & Execution Commands

```
javac ArrayDeque.java  
java ArrayDeque
```

Output Screenshot

```
PS C:\Users\Student> java ArrayDeque  
After addFirst(): [Priya, Rahul]  
After addLast(): [Priya, Rahul, Arjun, Sneha]  
After offerFirst(): [Kiran, Priya, Rahul, Arjun, Sneha]  
After offerLast(): [Kiran, Priya, Rahul, Arjun, Sneha, Meena]  
First element using peekFirst(): Kiran  
Last element using peekLast(): Meena  
Removed first element using pollFirst(): Kiran  
After pollFirst(): [Priya, Rahul, Arjun, Sneha, Meena]  
Removed last element using pollLast(): Meena  
After pollLast(): [Priya, Rahul, Arjun, Sneha]  
PS C:\Users\Student>
```

Result

The ArrayDeque methods were successfully demonstrated.

Program 2: ArrayList

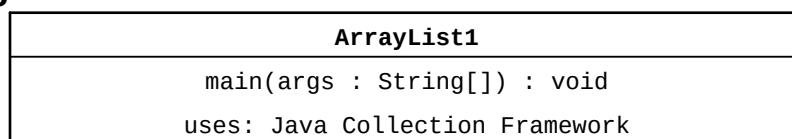
Aim / Objective

To demonstrate commonly used ArrayList methods for adding, accessing, updating, removing, searching, sorting, and clearing elements.

Algorithm / Logic

1. Create an ArrayList of fruits.
2. Add elements and insert an element at a specified index.
3. Access and update elements using get() and set().
4. Remove elements using index and object.
5. Check contains(), size(), and isEmpty().
6. Find first and last occurrences using indexOf() and lastIndexOf().
7. Sort the list and clear it.

UML Class Diagram



Source Code

```
import java.util.ArrayList;  
import java.util.Comparator;  
  
class ArrayList1 {  
  
    public static void main(String[] args) {  
  
        // Creating an ArrayList  
        ArrayList<String> fruits = new ArrayList<>();  
  
        // add(E e)  
        fruits.add("Apple");  
        fruits.add("Mango");  
        fruits.add("Banana");  
        fruits.add("Orange");  
  
        System.out.println("After add(): " + fruits);  
  
        // add(int index, E element)  
        fruits.add(2, "Grapes");  
        System.out.println("After add(index, element): " + fruits);  
    }  
}
```

```

// get(int index)
System.out.println("Element at index 1: " + fruits.get(1));

// set(int index, E element)
fruits.set(1, "Pineapple");
System.out.println("After set(): " + fruits);

// remove(int index)
fruits.remove(3);
System.out.println("After remove(index): " + fruits);

// remove(Object o)
fruits.remove("Apple");
System.out.println("After remove(object): " + fruits);

// contains(Object o)
System.out.println("Contains Banana? " + fruits.contains("Banana"));

// size()
System.out.println("Size of ArrayList: " + fruits.size());

// isEmpty()
System.out.println("Is ArrayList empty? " + fruits.isEmpty());

// indexOf(Object o)
fruits.add("Banana");
System.out.println("First index of Banana: " + fruits.indexOf("Banana"));

// lastIndexOf(Object o)
System.out.println("Last index of Banana: " + fruits.lastIndexOf("Banana"));

// sort(Comparator)
fruits.sort(Comparator.naturalOrder());
System.out.println("After sorting: " + fruits);

// clear()
fruits.clear();
System.out.println("After clear(): " + fruits);

// Checking isEmpty() again
System.out.println("Is ArrayList empty now? " + fruits.isEmpty());
}
}

```

Compilation & Execution Commands

```

javac ArrayList1.java
java ArrayList1

```

Output Screenshot

```

PS C:\Users\Student> java ArrayList1
After add(): [Apple, Mango, Banana, Orange]
After add(index, element): [Apple, Mango, Grapes, Banana, Orange]
Element at index 1: Mango
After set(): [Apple, Pineapple, Grapes, Banana, Orange]
After remove(index): [Apple, Pineapple, Grapes, Orange]
After remove(object): [Pineapple, Grapes, Orange]
Contains Banana? false
Size of ArrayList: 3
Is ArrayList empty? false
First index of Banana: 3
Last index of Banana: 3
After sorting: [Banana, Grapes, Pineapple, Orange]
After clear(): []
Is ArrayList empty now? true
PS C:\Users\Student>

```

Result

The ArrayList methods were successfully demonstrated.

Program 3: HashMap

Aim / Objective

To demonstrate commonly used HashMap methods for storing and managing key-value pairs.

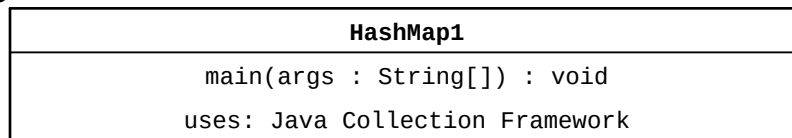
Algorithm / Logic

1. Create a HashMap of student IDs and names.
2. Add entries using put().
3. Retrieve and search values using get(), containsKey(), containsValue(), and getOrDefault().
4. Display keys, values, and entries.

5. Check size and empty status.

6. Remove an entry, clear the map, and check isEmpty().

UML Class Diagram



Source Code

```
import java.util.HashMap;

class HashMap1 {

    public static void main(String[] args) {

        // Creating a HashMap
        HashMap<Integer, String> students = new HashMap<>();

        // put(K key, V value)
        students.put(101, "Rahul");
        students.put(102, "Priya");
        students.put(103, "Arjun");
        students.put(104, "Sneha");

        System.out.println("After put(): " + students);

        // get(Object key)
        System.out.println("Student with ID 102: " + students.get(102));

        // containsKey(Object key)
        System.out.println("Contains key 103? "
            + students.containsKey(103));

        // containsValue(Object value)
        System.out.println("Contains value Arjun? "
            + students.containsValue("Arjun"));

        // getOrDefault(Object key, V defaultValue)
        System.out.println("Student with ID 105: "
            + students.getOrDefault(105, "Not Found"));

        // keySet()
        System.out.println("Keys: " + students.keySet());

        // values()
        System.out.println("Values: " + students.values());

        // entrySet()
        System.out.println("Key-Value pairs: " + students.entrySet());

        // size()
        System.out.println("Size of HashMap: " + students.size());

        // remove(Object key)
        students.remove(103);
        System.out.println("After remove(103): " + students);

        // isEmpty()
        System.out.println("Is HashMap empty? " + students.isEmpty());

        // clear()
        students.clear();
        System.out.println("After clear(): " + students);

        // isEmpty() after clear
        System.out.println("Is HashMap empty now? " + students.isEmpty());
    }
}
```

Compilation & Execution Commands

```
javac HashMap1.java
java HashMap1
```

Output Screenshot

```

PS C:\Users\Student> java HashMap1
After put(): {101=Rahul, 102=Priya, 103=Arjun, 104=Sneha}
Student with ID 102: Priya
Contains key 103? true
Contains value Arjun? true
Student with ID 105: Not Found
Keys: [101, 102, 103, 104]
Values: [Rahul, Priya, Arjun, Sneha]
Key-Value pairs: [101=Rahul, 102=Priya, 103=Arjun, 104=Sneha]
Size of HashMap: 4
After remove(103): {101=Rahul, 102=Priya, 104=Sneha}
Is HashMap empty? false
After clear(): {}
Is HashMap empty now? true
PS C:\Users\Student>

```

Result

The HashMap methods were successfully demonstrated.

Program 4: HashSet

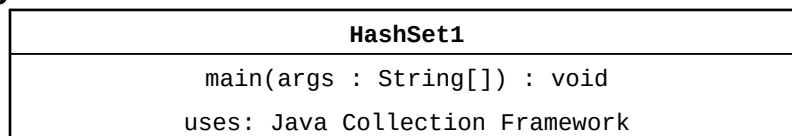
Aim / Objective

To demonstrate basic HashSet operations and duplicate prevention.

Algorithm / Logic

1. Create a HashSet of languages.
2. Add Java, Python, C++, and a duplicate Java.
3. Check membership using contains().
4. Display the set size.
5. Remove Python and check isEmpty().
6. Clear the set and check isEmpty() again.

UML Class Diagram



Source Code

```

import java.util.HashSet;

class HashSet1 {

    public static void main(String[] args) {

        // Creating a HashSet
        HashSet<String> languages = new HashSet<>();

        // add(E e)
        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("Java");    // Duplicate, will not be added

        System.out.println("After add(): " + languages);

        // contains(Object o)
        System.out.println("Contains Java? " + languages.contains("Java"));
        System.out.println("Contains HTML? " + languages.contains("HTML"));

        // size()
        System.out.println("Size of HashSet: " + languages.size());

        // remove(Object o)
        languages.remove("Python");
        System.out.println("After remove(): " + languages);

        // isEmpty()
        System.out.println("Is HashSet empty? " + languages.isEmpty());

        // clear()
        languages.clear();
        System.out.println("After clear(): " + languages);
        // isEmpty() after clear
    }
}

```

```

        System.out.println("Is HashSet empty now? " + languages.isEmpty());
    }
}

```

Compilation & Execution Commands

```

javac HashSet1.java
java HashSet1

```

Output Screenshot

```

PS C:\Users\Student> java HashSet1
After add(): [Java, C++, Python]
Contains Java? true
Contains HTML? false
Size of HashSet: 3
After remove(): [Java, C++]
Is HashSet empty? false
After clear(): []
Is HashSet empty now? true
PS C:\Users\Student>

```

Result

The HashSet methods were successfully demonstrated, including duplicate prevention.

Program 5: Hashtable

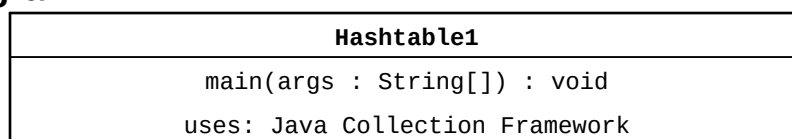
Aim / Objective

To demonstrate Hashtable methods and traversal using Enumeration.

Algorithm / Logic

1. Create a Hashtable of employee IDs and names.
2. Add key-value pairs using put().
3. Retrieve and search entries using get(), containsKey(), and containsValue().
4. Traverse keys using keys() and Enumeration.
5. Traverse values using elements() and Enumeration.
6. Display size, remove an entry, and check isEmpty().

UML Class Diagram



Source Code

```

import java.util.Hashtable;
import java.util.Enumeration;

class Hashtable1 {

    public static void main(String[] args) {

        // Creating a Hashtable
        Hashtable<Integer, String> employees = new Hashtable<>();

        // put(K key, V value)
        employees.put(101, "Rahul");
        employees.put(102, "Priya");
        employees.put(103, "Arjun");
        employees.put(104, "Sneha");

        System.out.println("After put(): " + employees);

        // get(Object key)
        System.out.println("Employee with ID 102: "
            + employees.get(102));

        // containsKey(Object key)
        System.out.println("Contains key 103? "

```

```

        + employees.containsKey(103));

// containsValue(Object value)
System.out.println("Contains value Rahul? "
    + employees.containsValue("Rahul"));

// keys()
System.out.println("Keys:");
Enumeration<Integer> keyList = employees.keys();

while (keyList.hasMoreElements()) {
    System.out.println(keyList.nextElement());
}

// elements()
System.out.println("Values:");
Enumeration<String> valueList = employees.elements();

while (valueList.hasMoreElements()) {
    System.out.println(valueList.nextElement());
}

// size()
System.out.println("Size of Hashtable: "
    + employees.size());

// remove(Object key)
employees.remove(103);
System.out.println("After remove(103): "
    + employees);

// isEmpty()
System.out.println("Is Hashtable empty? "
    + employees.isEmpty());
    }
}

```

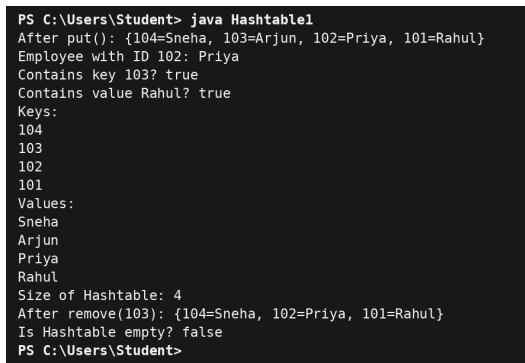
Compilation & Execution Commands

```

javac Hashtable1.java
java Hashtable1

```

Output Screenshot



```

PS C:\Users\Student> java Hashtable1
After put(): {104=Sneha, 103=Arjun, 102=Priya, 101=Rahul}
Employee with ID 102: Priya
Contains key 103? true
Contains value Rahul? true
Keys:
104
103
102
101
Values:
Sneha
Arjun
Priya
Rahul
Size of Hashtable: 4
After remove(103): {104=Sneha, 102=Priya, 101=Rahul}
Is Hashtable empty? false
PS C:\Users\Student>

```

Result

The Hashtable methods and Enumeration traversal were successfully demonstrated.

Program 6: LinkedHashMap

Aim / Objective

To demonstrate common LinkedHashMap operations while maintaining insertion order.

Algorithm / Logic

1. Create a LinkedHashMap of employee IDs and names.
2. Add key-value pairs using put().
3. Retrieve a value using get().
4. Check a key using containsKey().
5. Display keys, values, and entries.
6. Remove one employee and display the updated map.

UML Class Diagram

LinkedListHashMap1
<pre>main(args : String[]) : void uses: Java Collection Framework</pre>

Source Code

```
import java.util.HashMap;

class LinkedListHashMap1 {

    public static void main(String[] args) {

        // Creating a HashMap
        HashMap<Integer, String> employees = new HashMap<>();

        // put(K key, V value)
        employees.put(101, "Rahul");
        employees.put(102, "Priya");
        employees.put(103, "Arjun");
        employees.put(104, "Sneha");

        System.out.println("After put(): " + employees);

        // get(Object key)
        System.out.println("Employee with ID 102: "
            + employees.get(102));

        // containsKey(Object key)
        System.out.println("Contains key 103? "
            + employees.containsKey(103));

        // keySet()
        System.out.println("Keys: " + employees.keySet());

        // values()
        System.out.println("Values: " + employees.values());

        // entrySet()
        System.out.println("Key-Value pairs: "
            + employees.entrySet());

        // remove(Object key)
        employees.remove(103);
        System.out.println("After remove(103): "
            + employees);
    }
}
```

Compilation & Execution Commands

```
javac LinkedListHashMap1.java
java LinkedListHashMap1
```

Output Screenshot

```
PS C:\Users\Student> java LinkedListHashMap1
After put(): {101=Rahul, 102=Priya, 103=Arjun, 104=Sneha}
Employee with ID 102: Priya
Contains key 103? true
Keys: [101, 102, 103, 104]
Values: [Rahul, Priya, Arjun, Sneha]
Key-Value pairs: [101=Rahul, 102=Priya, 103=Arjun, 104=Sneha]
After remove(103): {101=Rahul, 102=Priya, 104=Sneha}
PS C:\Users\Student>
```

Result

The HashMap methods were successfully demonstrated.

Program 7: HashSet

Aim / Objective

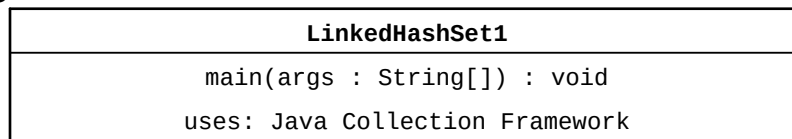
To demonstrate HashSet operations, duplicate prevention, and insertion order.

Algorithm / Logic

1. Create a HashSet of subjects.

2. Add Java, Python, C++, and a duplicate Java.
3. Check membership using contains().
4. Display size and remove Python.
5. Clear the set and display its size.

UML Class Diagram



Source Code

```
import java.util.LinkedHashSet;

class LinkedHashSet1 {

    public static void main(String[] args) {

        // Creating a LinkedHashSet
        LinkedHashSet<String> subjects = new LinkedHashSet<>();

        // add(E e)
        subjects.add("Java");
        subjects.add("Python");
        subjects.add("C++");
        subjects.add("Java"); // Duplicate, will not be added

        System.out.println("After add(): " + subjects);

        // contains(Object o)
        System.out.println("Contains Java? " + subjects.contains("Java"));
        System.out.println("Contains HTML? " + subjects.contains("HTML"));

        // size()
        System.out.println("Size of LinkedHashSet: " + subjects.size());

        // remove(Object o)
        subjects.remove("Python");
        System.out.println("After remove(): " + subjects);

        // clear()
        subjects.clear();
        System.out.println("After clear(): " + subjects);

        // Checking size after clear
        System.out.println("Size after clear(): " + subjects.size());
    }
}
```

Compilation & Execution Commands

```
javac LinkedHashSet1.java
java LinkedHashSet1
```

Output Screenshot

```
PS C:\Users\Student> java LinkedHashSet1
After add(): [Java, Python, C++]
Contains Java? true
Contains HTML? false
Size of LinkedHashSet: 3
After remove(): [Java, C++]
After clear(): []
Size after clear(): 0
PS C:\Users\Student>
```

Result

The LinkedHashSet methods were successfully demonstrated.

Program 8: LinkedList

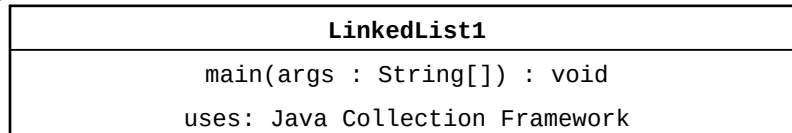
Aim / Objective

To demonstrate common LinkedList operations for list and queue-style manipulation.

Algorithm / Logic

1. Create a LinkedList of cities.
2. Add elements and add at the first and last positions.
3. Access elements using get(), getFirst(), and getLast().
4. Remove elements using index, object, first, and last.
5. Insert an element using offer().
6. View the head using peek().
7. Remove the head using poll().

UML Class Diagram



Source Code

```
import java.util.LinkedList;

class LinkedList1 {

    public static void main(String[] args) {

        // Creating a LinkedList
        LinkedList<String> cities = new LinkedList<>();

        // add(E e)
        cities.add("Hyderabad");
        cities.add("Chennai");
        cities.add("Mumbai");

        System.out.println("After add(): " + cities);

        // addFirst(E e)
        cities.addFirst("Delhi");
        System.out.println("After addFirst(): " + cities);

        // addLast(E e)
        cities.addLast("Kolkata");
        System.out.println("After addLast(): " + cities);

        // get(int index)
        System.out.println("Element at index 2: " + cities.get(2));

        // getFirst()
        System.out.println("First element: " + cities.getFirst());

        // getLast()
        System.out.println("Last element: " + cities.getLast());

        // remove(int index)
        cities.remove(2);
        System.out.println("After remove(index): " + cities);

        // remove(Object o)
        cities.remove("Chennai");
        System.out.println("After remove(object): " + cities);

        // removeFirst()
        String first = cities.removeFirst();
        System.out.println("Removed first element: " + first);
        System.out.println("After removeFirst(): " + cities);

        // removeLast()
        String last = cities.removeLast();
        System.out.println("Removed last element: " + last);
        System.out.println("After removeLast(): " + cities);

        // offer(E e)
        cities.offer("Pune");
        System.out.println("After offer(): " + cities);

        // peek()
        System.out.println("Head using peek(): " + cities.peek());
        System.out.println("List after peek(): " + cities);
        // poll()
```

```

        System.out.println("Removed head using poll(): " + cities.poll());
        System.out.println("List after poll(): " + cities);
    }
}

```

Compilation & Execution Commands

```

javac LinkedList1.java
java LinkedList1

```

Output Screenshot

```

PS C:\Users\Student> java LinkedList1
After add(): [Hyderabad, Chennai, Mumbai]
After addFirst(): [Delhi, Hyderabad, Chennai, Mumbai]
After addLast(): [Delhi, Hyderabad, Chennai, Mumbai, Kolkata]
Element at index 2: Chennai
First element: Delhi
Last element: Kolkata
After remove(index): [Delhi, Hyderabad, Mumbai, Kolkata]
After remove(object): [Delhi, Hyderabad, Mumbai, Kolkata]
Removed first element: Delhi
After removeFirst(): [Hyderabad, Mumbai, Kolkata]
Removed last element: Kolkata
After removeLast(): [Hyderabad, Mumbai]
After offer(): [Hyderabad, Mumbai, Pune]
Head using peek(): Hyderabad
List after peek(): [Hyderabad, Mumbai, Pune]
Removed head using poll(): Hyderabad
List after poll(): [Mumbai, Pune]
PS C:\Users\Student>

```

Result

The LinkedList methods were successfully demonstrated.

Program 9: PriorityQueue

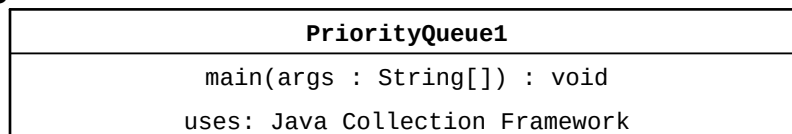
Aim / Objective

To demonstrate common PriorityQueue operations including insertion, retrieval, searching, removal, and size checking.

Algorithm / Logic

1. Create a PriorityQueue of integers.
2. Add elements using add() and offer().
3. View the head using peek().
4. Check membership using contains().
5. Display the queue size.
6. Remove an element using remove() and remove the head using poll().

UML Class Diagram



Source Code

```

import java.util.PriorityQueue;

class PriorityQueue1 {

    public static void main(String[] args) {

        // Creating a PriorityQueue
        PriorityQueue<Integer> tasks = new PriorityQueue<>();

        // add(E e)
        tasks.add(40);
        tasks.add(10);
        tasks.add(30);

        System.out.println("After add(): " + tasks);

        // offer(E e)
        tasks.offer(20);
        System.out.println("After offer(): " + tasks);

        // peek()
        System.out.println("Head using peek(): " + tasks.peek());
    }
}

```

```

        System.out.println("Queue after peek(): " + tasks);

        // contains(Object o)
        System.out.println("Contains 30? " + tasks.contains(30));
        System.out.println("Contains 50? " + tasks.contains(50));

        // size()
        System.out.println("Size of PriorityQueue: " + tasks.size());

        // remove(Object o)
        tasks.remove(30);
        System.out.println("After remove(30): " + tasks);

        // poll()
        System.out.println("Removed head using poll(): " + tasks.poll());
        System.out.println("Queue after poll(): " + tasks);

        // Checking size again
        System.out.println("Final size: " + tasks.size());
    }
}

```

Compilation & Execution Commands

```

javac PriorityQueue1.java
java PriorityQueue1

```

Output Screenshot

```

PS C:\Users\Student> java PriorityQueue1
After add(): [10, 40, 30]
After offer(): [10, 20, 30, 40]
Head using peek(): 10
Queue after peek(): [10, 20, 30, 40]
Contains 30? true
Contains 50? false
Size of PriorityQueue: 4
After remove(30): [10, 20, 40]
Removed head using poll(): 10
Queue after poll(): [20, 40]
Final size: 2
PS C:\Users\Student>

```

Result

The PriorityQueue methods were successfully demonstrated.

Program 10: Stack

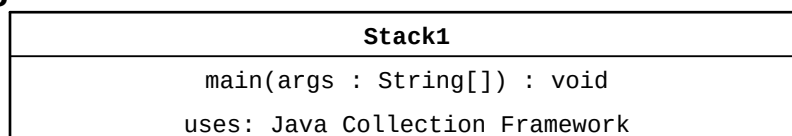
Aim / Objective

To demonstrate Stack operations using push, peek, search, pop, and empty.

Algorithm / Logic

1. Create a Stack of book names.
2. Push four books onto the stack.
3. View the top element using peek().
4. Find positions using search().
5. Remove the top element using pop().
6. Check empty(), remove remaining elements, and check empty() again.

UML Class Diagram



Source Code

```

import java.util.Stack;

class Stack1 {

    public static void main(String[] args) {
        // Creating a Stack
    }
}

```

```

Stack<String> books = new Stack<>();

// push(E item)
books.push("Java");
books.push("Python");
books.push("C++");
books.push("DBMS");

System.out.println("After push(): " + books);

// peek()
System.out.println("Top element using peek(): " + books.peek());
System.out.println("Stack after peek(): " + books);

// search(Object o)
System.out.println("Position of Python: " + books.search("Python"));
System.out.println("Position of Java: " + books.search("Java"));

// pop()
String removed = books.pop();
System.out.println("Removed element using pop(): " + removed);
System.out.println("Stack after pop(): " + books);

// empty()
System.out.println("Is Stack empty? " + books.empty());

// Pop remaining elements
books.pop();
books.pop();
books.pop();

System.out.println("Stack after removing all elements: " + books);

// Checking empty again
System.out.println("Is Stack empty now? " + books.empty());
}
}

```

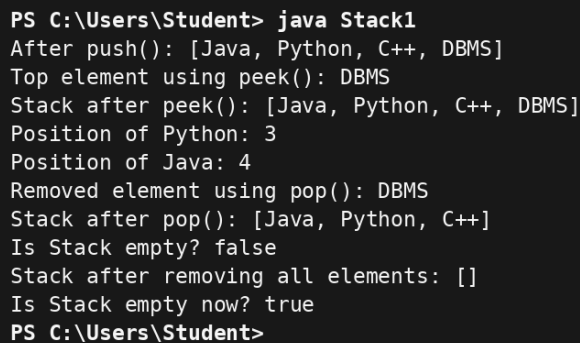
Compilation & Execution Commands

```

javac Stack1.java
java Stack1

```

Output Screenshot



```

PS C:\Users\Student> java Stack1
After push(): [Java, Python, C++, DBMS]
Top element using peek(): DBMS
Stack after peek(): [Java, Python, C++, DBMS]
Position of Python: 3
Position of Java: 4
Removed element using pop(): DBMS
Stack after pop(): [Java, Python, C++]
Is Stack empty? false
Stack after removing all elements: []
Is Stack empty now? true
PS C:\Users\Student>

```

Result

The Stack methods were successfully demonstrated.

Program 11: TreeMap

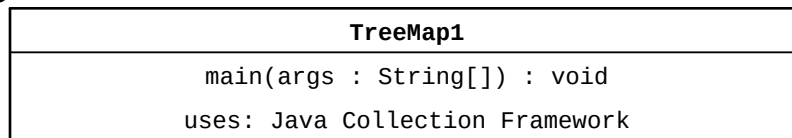
Aim / Objective

To demonstrate TreeMap operations and navigation based on sorted keys.

Algorithm / Logic

1. Create a TreeMap of student IDs and names.
2. Add key-value pairs.
3. Retrieve and search entries using get(), containsKey(), and containsValue().
4. Display firstKey() and lastKey().
5. Use higherKey(), lowerKey(), ceilingKey(), and floorKey().
6. Display entries and remove one key.

UML Class Diagram



Source Code

```
import java.util.TreeMap;

class TreeMap1 {

    public static void main(String[] args) {

        // Creating a TreeMap
        TreeMap<Integer, String> students = new TreeMap<>();

        // put(K key, V value)
        students.put(103, "Arjun");
        students.put(101, "Rahul");
        students.put(105, "Sneha");
        students.put(102, "Priya");
        students.put(104, "Kiran");

        System.out.println("After put(): " + students);

        // get(Object key)
        System.out.println("Student with ID 102: "
            + students.get(102));

        // containsKey(Object key)
        System.out.println("Contains key 103? "
            + students.containsKey(103));

        // containsValue(Object value)
        System.out.println("Contains value Rahul? "
            + students.containsValue("Rahul"));

        // firstKey()
        System.out.println("First key: " + students.firstKey());

        // lastKey()
        System.out.println("Last key: " + students.lastKey());

        // higherKey(K key)
        System.out.println("Key higher than 102: "
            + students.higherKey(102));

        // lowerKey(K key)
        System.out.println("Key lower than 102: "
            + students.lowerKey(102));

        // ceilingKey(K key)
        System.out.println("Ceiling key of 103: "
            + students.ceilingKey(103));

        // floorKey(K key)
        System.out.println("Floor key of 103: "
            + students.floorKey(103));

        // entrySet()
        System.out.println("Key-Value pairs: "
            + students.entrySet());

        // remove(Object key)
        students.remove(104);
        System.out.println("After remove(104): "
            + students);
    }
}
```

Compilation & Execution Commands

```
javac TreeMap1.java
java TreeMap1
```

Output Screenshot

```

PS C:\Users\Student> java TreeMap1
After put(): {101=Rahul, 102=Priya, 103=Arjun, 104=Kiran, 105=Sneha}
Student with ID 102: Priya
Contains key 103? true
Contains value Rahul? true
First key: 101
Last key: 105
Key higher than 102: 103
Key lower than 102: 101
Ceiling key of 103: 103
Floor key of 103: 103
Key-Value pairs: [101=Rahul, 102=Priya, 103=Arjun, 104=Kiran, 105=Sneha]
After remove(104): {101=Rahul, 102=Priya, 103=Arjun, 105=Sneha}
PS C:\Users\Student>

```

Result

The TreeMap methods were successfully demonstrated.

Program 12: TreeSet

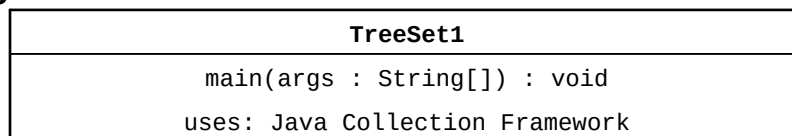
Aim / Objective

To demonstrate TreeSet operations, sorted order, searching, and boundary removal.

Algorithm / Logic

1. Create a TreeSet of integers.
2. Add elements and display sorted order.
3. Check membership using contains().
4. Retrieve first and last elements.
5. Use higher(), lower(), ceiling(), and floor().
6. Remove an element and then use pollFirst() and pollLast().

UML Class Diagram



Source Code

```

import java.util.TreeSet;

class TreeSet1 {

    public static void main(String[] args) {

        // Creating a TreeSet
        TreeSet<Integer> numbers = new TreeSet<>();

        // add(E e)
        numbers.add(40);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        numbers.add(50);

        System.out.println("After add(): " + numbers);

        // contains(Object o)
        System.out.println("Contains 30? " + numbers.contains(30));
        System.out.println("Contains 60? " + numbers.contains(60));

        // first()
        System.out.println("First element: " + numbers.first());

        // last()
        System.out.println("Last element: " + numbers.last());

        // higher(E e)
        System.out.println("Element higher than 30: " + numbers.higher(30));

        // lower(E e)
        System.out.println("Element lower than 30: " + numbers.lower(30));

        // ceiling(E e)

```

```

        System.out.println("Ceiling of 25: " + numbers.ceiling(25));

        // floor(E e)
        System.out.println("Floor of 25: " + numbers.floor(25));

        // remove(Object o)
        numbers.remove(20);
        System.out.println("After remove(20): " + numbers);

        // pollFirst()
        System.out.println("Removed first element: " + numbers.pollFirst());
        System.out.println("After pollFirst(): " + numbers);

        // pollLast()
        System.out.println("Removed last element: " + numbers.pollLast());
        System.out.println("After pollLast(): " + numbers);
    }
}

```

Compilation & Execution Commands

```

javac TreeSet1.java
java TreeSet1

```

Output Screenshot

```

PS C:\Users\Student> java TreeSet1
After add(): [10, 20, 30, 40, 50]
Contains 30? true
Contains 60? false
First element: 10
Last element: 50
Element higher than 30: 40
Element lower than 30: 20
Ceiling of 25: 30
Floor of 25: 20
After remove(20): [10, 30, 40, 50]
Removed first element: 10
After pollFirst(): [30, 40, 50]
Removed last element: 50
After pollLast(): [30, 40]
PS C:\Users\Student>

```

Result

The TreeSet methods were successfully demonstrated.

Program 13: Vector

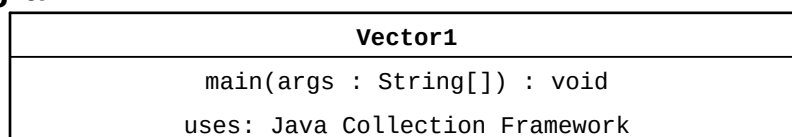
Aim / Objective

To demonstrate common Vector methods for adding, accessing, updating, removing, sizing, capacity, and searching.

Algorithm / Logic

1. Create a Vector of subjects.
2. Add elements using add() and addElement().
3. Access and update an element using get() and set().
4. Remove elements using remove() and removeElement().
5. Display size and capacity.
6. Check membership using contains().

UML Class Diagram



Source Code

```

import java.util.Vector;

class Vector1 {

    public static void main(String[] args) {

        // Creating a Vector
        Vector<String> subjects = new Vector<>();
    }
}

```

```

// add(E e)
subjects.add("Java");
subjects.add("Python");
subjects.add("C++");

System.out.println("After add(): " + subjects);

// addElement(Object obj)
subjects.addElement("DBMS");
System.out.println("After addElement(): " + subjects);

// get(int index)
System.out.println("Element at index 1: " + subjects.get(1));

// set(int index, E element)
subjects.set(2, "C");
System.out.println("After set(): " + subjects);

// remove(int index)
subjects.remove(1);
System.out.println("After remove(index): " + subjects);

// removeElement(Object obj)
subjects.removeElement("DBMS");
System.out.println("After removeElement(): " + subjects);

// size()
System.out.println("Size of Vector: " + subjects.size());

// capacity()
System.out.println("Capacity of Vector: " + subjects.capacity());

// contains(Object o)
System.out.println("Contains Java? " + subjects.contains("Java"));
System.out.println("Contains Python? " + subjects.contains("Python"));
}

```

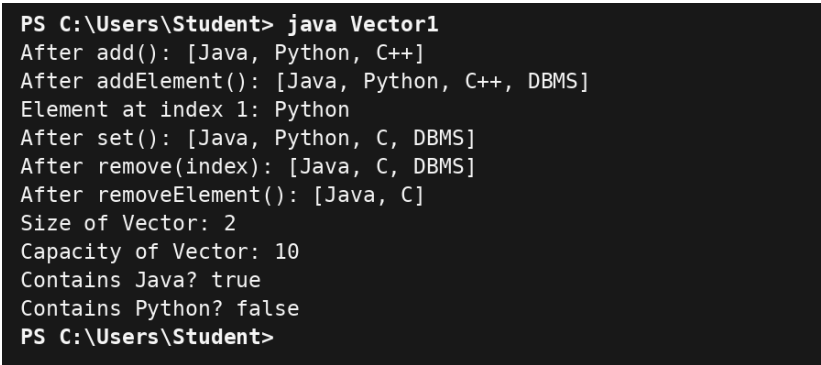
Compilation & Execution Commands

```

javac Vector1.java
java Vector1

```

Output Screenshot



```

PS C:\Users\Student> java Vector1
After add(): [Java, Python, C++]
After addElement(): [Java, Python, C++, DBMS]
Element at index 1: Python
After set(): [Java, Python, C, DBMS]
After remove(index): [Java, C, DBMS]
After removeElement(): [Java, C]
Size of Vector: 2
Capacity of Vector: 10
Contains Java? true
Contains Python? false
PS C:\Users\Student>

```

Result

The Vector methods were successfully demonstrated.